



LEDGEROPS

SECURITY ASSESSMENT REPORT

Solve3

May 13, 2026

TABLE OF CONTENTS

TABLE OF CONTENTS 1

EXECUTIVE SUMMARY 2

ENGAGEMENT SCOPE 4

 SCOPE OF WORK 4

Testing Scope 4

 ASSUMPTIONS 4

 TIMELINE 4

TECHNICAL FINDINGS OVERVIEW 5

EXECUTIVE SUMMARY

This report presents the findings from a security assessment conducted on the ve(3,3) protocol implementation on Solana, focusing on the Solve3 Finance smart contract ecosystem. The assessment reviewed the protocol's core smart contracts, including token minting logic, vote-escrow mechanisms, gauge voting, fee distribution, and cross-program invocation (CPI) interactions within the Anchor framework.

The review aimed to evaluate the platform's security posture, identify vulnerabilities that could impact protocol integrity, and provide recommendations to strengthen the security and reliability of the system prior to production deployment.

Scope of Testing

The security assessment covered the following components:

Smart Contracts

- **solve-token:** SPL-style token implementation responsible for minting, transferring, and burning the SOLVE token.
- **solve3:** Core ve(3,3) protocol logic, including lock creation, gauge voting, rewards distribution, and fee accounting.
- **vesolve-token:** veSOLVE NFT contract responsible for minting, updating, and managing vote-escrow positions.

The review included analysis of program logic, PDA derivation, cross-program invocation trust boundaries, access control mechanisms, and economic accounting flows.

Risk Summary

A total of **0 Critical, 4 High, 2 Medium, 1 Low, and 1 Informational findings** were identified during the assessment.

The high-severity findings primarily involve authorization and protocol logic issues that could allow unauthorized minting of vote-escrow NFTs or disrupt the intended governance and reward distribution mechanisms. Additional findings relate to incomplete fee accounting logic, inconsistencies in PDA derivation for vote escrow locks, and configuration validation weaknesses during program initialization.

If exploited, these vulnerabilities could lead to:

- Unauthorized minting of veSOLVE governance tokens
- Broken or inconsistent voting and bribe distribution mechanisms
- Incorrect fee accounting and reward distribution
- Potential misuse of protocol initialization parameters
- Operational risks related to administrative key management

Addressing these issues will help ensure the protocol maintains the intended economic guarantees of the ve(3,3) model and prevents unauthorized manipulation of governance or reward flows.

Important Note

To maintain a secure environment, Solve3 Finance should ensure that all smart contract logic is thoroughly tested, validated, and monitored prior to deployment. Regular security reviews, dependency updates, and monitoring of administrative privileges are recommended to maintain a strong security posture over time.

REPORT CARD

Smart Contracts

Finding	Status	Severity
Missing CPI Caller Validation in vesolve-token Mint	Closed	High
VeLock PDA Seed Mismatch in Vote, ClaimBribes, and ClaimFees	Closed	High
Gauge total_votes Never Updated in vote()	Closed	High
Fee Accounting Not Updated by add_collected_fees	Closed	High
Unvalidated vesolve_token_program in initialize	Closed	Medium
claim_fees Uses protocol.total_locked as Total Voting Power	Closed	Medium
SOLVE Mint Authority Held by Deployer Keypair	Closed	Low
Backend Admin Keypair and Replay / Idempotency Risk	Closed	Informational

ENGAGEMENT SCOPE

SCOPE OF WORK

This report is part of the security assessment conducted during the period outlined in the timeline and covers the security assessment scopes of concern for Solve3.

TESTING SCOPE

Here's pertinent information about the testing scope. We believe you'll find this information about the testing scope to be pertinent, as that is exactly the reason it was judged worthy of inclusion, in this particular portion of the report narrative.

ASSUMPTIONS

NDA and rules of engagement have been signed based on the information provided and gathered during information gathering phase and the company name is Solve3.

TIMELINE

The timeline of the testing is included below:

Test Type	Start Date/Time	End Date/Time
Security Assessment	February 16, 2026	February 27, 2026

TECHNICAL FINDINGS OVERVIEW



The Solve3 web application contains vulnerabilities that could impact both the security of the application and the reputation of Solve3 Finance. Addressing these issues is critical to ensuring a secure production environment and maintaining user trust.

LedgerOps strongly recommends remediating these vulnerabilities prior to deployment in a live environment. Failure to do so could expose the platform to unauthorized access, data breaches, and operational disruptions.

For detailed technical information on all identified risks, including their severity, evidence, and recommended remediation steps, please refer to the Technical Findings section of this report. Below is a summary of the vulnerabilities identified during the assessment.

Finding Name	Risk Level	CVSS
High Risk Findings		
Missing CPI Caller Validation in vesolve-token Mint	High	CVSS 9.1
VeLock PDA Seed Mismatch in Vote, ClaimBribes, and ClaimFees	High	CVSS 8.2
Gauge total_votes Never Updated in vote()	High	CVSS 7.5
Fee Accounting Not Updated by add_collected_fees	High	CVSS 6.5
Medium Risk Findings		
Unvalidated vesolve_token_program in initialize	Medium	CVSS 5.3
claim_fees Uses protocol.total_locked as Total Voting Power	Medium	CVSS 4.3
Low Risk Findings		
SOLVE Mint Authority Held by Deployer Keypair	Low	CVSS 3.7
Informational Risk Findings		
Backend Admin Keypair and Replay / Idempotency Risk	Informational	CVSS 3.1

MISSING CPI CALLER VALIDATION IN VESOLVE- TOKEN MINT

High

SUMMARY

The vesolve-token program is intended to mint vote-escrow NFTs (veSOLVE) only through the solve3 program after a valid SOLVE token lock has been created. However, the current implementation does not properly validate the caller during cross-program invocation (CPI).

The mint instruction verifies that the ve33_program account matches the stored program identifier but does not require the program account to be a signer. On Solana, a program account passed directly by a client is not automatically validated as the true caller unless signer validation is enforced.

As a result, an attacker can directly invoke the mint_ve_nft instruction and bypass the intended CPI-only restriction.

IMPACT

Successful exploitation could allow attackers to mint arbitrary veSOLVE NFTs without locking any SOLVE tokens. This breaks the core assumptions of the ve(3,3) governance model and allows attackers to create artificial voting power.

Potential impacts include:

- Unauthorized minting of vote-escrow NFTs
- Manipulation of governance voting
- Illegitimate bribe or reward claims
- Disruption of protocol economics

EVIDENCE

The mint instruction validates only that the supplied ve33_program account matches the stored program identifier but does not verify that the program account is a signer.

Example pattern observed:

```
constraint = ve_mint_authority.ve33_program == ve33_program.key()
```

The instruction does not enforce a signer requirement such as:

```
require!(ve33_program.is_signer)
```

Similar patterns were identified in related instructions such as burn and update.

REMEDIATION

Implement strict caller validation within the vesolve-token program to ensure that minting can only occur through CPI from the authorized solve3 program.

Recommended approach:

- Require the ve33_program account to be a signer during the mint operation.
- Reject any direct client invocation that does not originate from the authorized program.
- Apply similar validation checks to related instructions (burn, update) where applicable.

CVSS 9.1

STATUS: REMEDIATED

May 13, 2026

The separate vesolve-token program was removed. veNFT creation is internal to vesolve and only exposed through lock lifecycle instructions.

VELOCK PDA SEED MISMATCH IN VOTE, CLAIMBRIBES, AND CLAIMFEES

High

SUMMARY

The VeLock account derivation used during lock creation is inconsistent with the derivation expected by several protocol instructions.

Locks are created using a three-seed PDA derivation:
[VE_LOCK_SEED, user.key(), nft_token_id]

However, several instructions that reference this lock derive the PDA using only two seeds:
[VE_LOCK_SEED, user.key()]

Because these derivations differ, the account expected by the vote and reward instructions cannot match the account created by the lock creation logic.

IMPACT

This inconsistency can break core protocol functionality by preventing the correct VeLock accounts from being referenced by governance and reward instructions.

Potential impacts include:

- Voting functionality failing or referencing incorrect accounts
- Bribe claims failing due to invalid account derivation
- Fee claims referencing non-existent lock accounts
- Inconsistent governance behavior

These issues undermine the protocol's core ve(3,3) voting and reward distribution mechanisms.

EVIDENCE

Lock creation instructions derive the VeLock account using three seeds:
[VE_LOCK_SEED, user.key(), nft_token_id]

Vote and claim instructions derive the account using two seeds:
[VE_LOCK_SEED, user.key()]

Since PDA derivation must match exactly, the accounts referenced in these instructions cannot correspond to the accounts created during lock initialization.

REMEDIATION

Ensure consistent PDA derivation across all instructions interacting with the VeLock account.

Possible remediation strategies include:

- Adopting a single-lock-per-user model using seeds [VE_LOCK_SEED, user], or
- Updating voting and reward instructions to accept a lock identifier (e.g., nft_token_id) and derive the PDA using [VE_LOCK_SEED, user, nft_token_id].

Protocol documentation should clearly define whether multiple locks per user are supported and how voting power is aggregated.

CVSS 8.2

STATUS: REMEDIATED

May 13, 2026

All lock-consuming instructions use the same 3-seed derivation and pass ve_token_id (or ve_token_id from instruction args) explicitly.

GAUGE TOTAL_VOTES NEVER UPDATED IN VOTE()

High

SUMMARY

The `vote()` instruction records user votes but does not update the `total_votes` value stored within gauge accounts. Gauge accounts are initialized with `total_votes = 0` and are never updated during the voting process.

A helper function (`update_gauge_votes`) exists within the codebase but is not invoked during the vote operation.

As a result, gauge vote totals remain zero even when users cast votes.

IMPACT

The protocol relies on `total_votes` to calculate reward and bribe distribution. Because this value remains zero, reward calculations may fail or behave incorrectly.

Potential impacts include:

- Bribe claims failing due to division by zero
- Incorrect reward distribution
- Non-functional gauge voting mechanics
- Governance and incentive mechanisms operating incorrectly

EVIDENCE

Within the `vote()` instruction, vote allocations are written to the `UserVote` account but no update occurs to any associated gauge accounts.

Example observation:

- `UserVote` state updated
- `Gauge.total_votes` remains unchanged

Later reward calculations rely on this value:

```
let total_gauge_votes = gauge_account.total_votes
```

If the value remains zero, reward calculations may fail or produce incorrect results.

REMEDIATION

Ensure gauge vote totals are updated whenever votes are cast or revoked.

Recommended solutions include:

- Updating `vote()` to modify the `total_votes` value for each gauge involved
- Passing gauge accounts as instruction inputs and updating them during vote execution
- Alternatively, implementing a reliable off-chain snapshot system and adding safeguards for zero-vote scenarios

CVSS 7.5

STATUS: REMEDIATED

May 13, 2026

Vote totals are tracked on the **epoch** account via `gauge_weights` (per-gauge weight and vote count). `vote()` updates these inline and refreshes aggregates.

FEE ACCOUNTING NOT UPDATED BY ADD_COLLECTED_FEES

High

SUMMARY

The `add_collected_fees()` instruction currently logs fee information but does not update any on-chain accounting state. As a result, the protocol does not record fee accrual within the accounts used by the `claim_fees()` instruction.

The `claim_fees()` function relies on `user_rewards.accumulated_fees` to determine the amount a user can claim. However, this value is initialized to zero and never increased by any instruction.

IMPACT

Because fee accrual is never recorded on-chain, users are unable to claim any fees even if fees are collected by the protocol.

Potential impacts include:

- Fee distribution mechanisms not functioning as intended
- Users receiving zero rewards despite protocol fee generation
- Misalignment between protocol behavior and user expectations

EVIDENCE

The `add_collected_fees()` instruction only logs a message and does not update protocol state variables or user reward accounts.

Example observation:

- `user_rewards.accumulated_fees` initialized to 0
- No instruction increases this value
- `claim_fees()` decreases the value when claims occur

Therefore, the effective claimable amount remains zero.

REMEDIATION

Implement proper fee accounting within the protocol to ensure collected fees are recorded and distributed correctly.

Recommended approaches include:

- Updating `add_collected_fees()` to record fee amounts within a protocol-level accounting structure
- Tracking fee distribution by epoch or voting period
- Ensuring that the `claim_fees()` instruction references a state variable that is correctly incremented when fees are collected

Clear documentation should also specify whether fee distribution is handled fully on-chain or partially through off-chain processes.

CVSS 6.5

STATUS: REMEDIATED

May 13, 2026

Removed the stub accounting path. Rewards are **vault-backed**: admin distributes tokens into a gauge vault; users claim proportional shares from that vault.

UNVALIDATED VESOLVE_TOKEN_PROGRAM IN INITIALIZE

Medium

SUMMARY

The initialize instruction accepts a `vesolve_token_program` account that is later used for cross-program invocation (CPI) when interacting with the veSOLVE NFT contract. However, this account is defined as an unchecked `AccountInfo` and is not validated to ensure that it corresponds to the intended vesolve-token program.

Because the program identifier is not verified during initialization, an incorrect or malicious program address could be stored within the protocol configuration.

Subsequent operations that rely on this stored program identifier could then execute CPI calls to an unintended or attacker-controlled program.

IMPACT

If a malicious or incorrect program identifier is stored during initialization, future CPI calls could be redirected to an attacker-controlled program.

Potential impacts include:

- Execution of unintended program logic during mint or burn operations
- Corruption of protocol state
- Loss of assets or protocol integrity if malicious instructions are executed

Although this risk primarily depends on who performs the initialization, validating program identifiers is considered a best practice for secure Solana program design.

EVIDENCE

The initialize instruction accepts the `vesolve_token_program` account with a `CHECK` annotation, indicating that Anchor does not perform validation.

Example pattern observed:

```
/// CHECK: This account is not validated  
pub vesolve_token_program: AccountInfo<'info>
```

The program identifier is then stored in protocol state without verification.

REMEDIATION

Validate the `vesolve_token_program` identifier during initialization to ensure it matches the intended program.

Recommended approaches include:

- Enforcing that the program identifier equals the known `vesolve-token` program ID
- Implementing an allowlist of trusted program identifiers
- Rejecting initialization if the provided program does not match the expected address

These controls ensure that future CPI calls are executed only against trusted programs.

CVSS 5.3

STATUS: REMEDIATED

May 13, 2026

No external `vesolve` program id is stored. `initialize` takes `mint_token`: Pubkey (the `SOLVE` SPL mint) only.

CLAIM_FEES USES TOTAL_LOCKED AS TOTAL VOTING POWER

Medium

SUMMARY

The `claim_fees()` instruction calculates a user's share of collected protocol fees using `protocol.total_locked` as a proxy for total voting power. However, the `ve(3,3)` model typically calculates voting power based on time-weighted locks, meaning voting power depends on both the amount of tokens locked and the duration of the lock.

Using the total amount of tokens locked rather than the actual time-weighted voting power results in an inaccurate representation of governance influence when distributing fees.

IMPACT

This simplified calculation can lead to inaccurate fee distribution among `veSOLVE` holders.

Potential impacts include:

- Users with shorter lock durations receiving a larger share of fees than intended
- Inconsistent reward distribution compared to the expected `ve(3,3)` economic model
- Reduced alignment between governance participation and reward incentives

While this issue does not directly enable exploitation, it introduces inconsistencies in the protocol's reward mechanics.

EVIDENCE

Within the `claim_fees()` implementation, the protocol uses the following logic:

```
let total_voting_power = protocol.total_locked;
```

The code comments acknowledge that this value is a simplified approximation and that true voting power should be derived from the lock structure.

However, no implementation of time-weighted voting power aggregation is present in the reviewed code.

REMEDIATION

Implement a more accurate calculation of total voting power when distributing fees.

Possible solutions include:

- Calculating voting power dynamically using each lock's remaining duration
- Maintaining a protocol-level aggregate of time-weighted voting power
- Updating voting power values whenever locks are created, extended, or withdrawn

Aligning the fee distribution logic with the intended `ve(3,3)` economic model will improve fairness and consistency across the protocol.

CVSS 4.3

STATUS: REMEDIATED

May 13, 2026

Claims use `epoch.total_voting_power`, derived from summed gauge weights after votes.
Per-user power comes from `lock.calculate_voting_power(current_time)` at vote time.

SOLVE MINT AUTHORITY HELD BY DEPLOYER KEYPAIR

Low

SUMMARY

The SOLVE token mint authority is currently assigned to the deployer's keypair during initialization rather than being controlled by a program-derived address (PDA) or multisignature authority. While this design pattern is not inherently insecure, it introduces operational risk by concentrating minting privileges in a single externally managed key.

If the deployer keypair is compromised, an attacker could mint arbitrary quantities of the SOLVE token. Conversely, if the keypair is lost, the protocol may lose the ability to mint additional tokens required for emissions or operational needs.

IMPACT

Potential impacts include:

- Unauthorized token minting if the deployer keypair is compromised
- Permanent loss of minting capability if the keypair is lost
- Centralized control over token supply

Although this configuration may be acceptable during early development or testing phases, production deployments typically require stronger key management controls.

EVIDENCE

The mint authority is set during token initialization to the signer account:

```
mint::authority = authority
```

Subsequent mint operations require the authority keypair to sign:

```
constraint = authority.key() == mint.mint_authority.unwrap()
```

No multisignature or program-controlled authority mechanism is implemented.

REMEDIATION

Consider implementing stronger mint authority controls for production environments.

Recommended approaches include:

- Using a multisignature wallet for mint authority
- Assigning mint authority to a program-derived address (PDA)
- Implementing governance-controlled minting mechanisms

Proper key management procedures and documented key rotation policies are also recommended.

CVSS 3.7

STATUS: REMEDIATED

May 13, 2026

Token program uses a PDA as mint authority (`mint::authority = mint.key()`, seeds `[b"mint"]`).
Minting requires `MintConfig::is_minter()`: either the config authority or a program id on the minter allowlist (detected via instructions `sysvar` for CPI).

BACKEND ADMIN KEYPAIR AND REPLAY / IDEMPOTENCY RISK

Informational

SUMMARY

The protocol relies on an off-chain backend service that holds the administrative keypair used to execute privileged instructions such as epoch updates, reward distribution, and emission adjustments. While this design is common in many blockchain protocols, it introduces operational risks related to key management and transaction replay protection.

If the backend administrative keypair were compromised, an attacker could execute privileged protocol instructions and manipulate governance or reward distribution processes. Additionally, if administrative transactions are not designed to be idempotent or protected against replay, duplicate execution could occur under certain conditions.

IMPACT

Potential impacts include:

- Unauthorized execution of administrative instructions if the keypair is compromised
- Manipulation of reward distributions or emission parameters
- Duplicate execution of administrative operations if replay protections are not enforced

These risks are primarily operational and depend on backend security practices rather than on-chain vulnerabilities.

EVIDENCE

Administrative instructions such as the following require only that the signer matches the stored protocol admin:

- `update_epoch`
- `distribute_to_pool`
- `add_collected_fees`
- `update_emission_rate`

The backend infrastructure manages the administrative keypair used to sign these transactions.

REMEDIATION

To reduce operational risk, the following best practices are recommended:

- Store the administrative keypair in secure infrastructure such as a hardware security module (HSM) or managed key vault
- Consider using multisignature authorization for critical administrative operations
- Implement replay protection or idempotent logic for administrative transactions
- Monitor administrative actions through logging and alerting mechanisms

These measures help reduce the risk of administrative key compromise or unintended protocol state changes.

CVSS 3.1

STATUS: REMEDIATED

May 13, 2026

Epoch advancement has time gating and unique epoch accounts per advance.